

CMPT 276: Code Review Report

Reviewer: Samantha Gan Reviewee: Krish Kaushik

Overview

For Assignment 4, I have reviewed two classes that Krish has contributed to: `PoliceEnemy.java` and `GameScreen.java`. I have verified each section of code to be Krish's by reviewing commit history, and ensuring his name is on the blame. Through investigation, I have found 4 code smells for Krish to refactor.

Smell 1: Code Duplication

Files: `PoliceEnemy.java` **Lines:** 142-154 and 187-199 **Commit:** eeac4fb

Duplicated `blockedX` and `blockedY` in alert/chasing state and the wander state. Both compute identical `hitW`, `hitTop`, `blockedX`, and `blockedY` values using the same four `isBarrier()` calls each time. Lines 142 - 154 and Lines 187 - 199 are the same.

```
boolean blockedX =
    lookup.isBarrier(newX - hitW, position.y - SIZE / 2f) ||
    lookup.isBarrier(newX + hitW, position.y - SIZE / 2f) ||
    lookup.isBarrier(newX - hitW, position.y - hitTop) ||
    lookup.isBarrier(newX + hitW, position.y - hitTop);
    if (!blockedX) position.x = newX;
boolean blockedY = ... if (!blockedY) position.y = newY; // Similar but Y
```

This can be fixed by putting the repeated code into a dedicated private method (eg. `applyBarrierCollision()`).

Smell 2: Method is too long

Files: `update()` method in `PoliceEnemy.java` **Lines:** 96 - 221 **Commit:** eeac4fb

The `update()` method in `PoliceEnemy.java` handles state, movement/collision and animation. This method is long, and is difficult to read and maintain.

```
public void update(float delta, float playerX, float playerY, BarrierLookup lookup) {
    // state machine
    // movement
    // wander
    // animation ...}
```

Having helper functions for each task can increase readability. By calling each helper in the `update()` method, it reduces the amount of code inside the method, and breaking it up into 3 digestible, separate chunks. `update()` serves as a clean coordinator.

Separate into different methods:

- `updateAlertState(float delta, float dist)`
- `updateMovement(float delta, float playerX, float playerY, BarrierLookup lookup)`
- `updateAnimation(float delta)`

Smell 3: Low Cohesion

Files: `render()` in `PoliceEnemy.java` **Lines:** 223-239 **Commit:** eeac4fb

The `render()` method mixes two different responsibilities: sprite drawing, and conditional logic that reads `alertState` to decide which indicator to display.

```
public void render(SpriteBatch batch) {
    float x = position.x - SIZE / 2f;
    float y = position.y - SIZE / 2f;
    if (lastDirection == Direction.RIGHT) {...}
    else {...}
    // draw ? or ! above head depending on state
    if (alertState == AlertState.ALERTED) {...}
    else if (alertState == AlertState.CHASING) {...}
}
```

A render method should only be presentational, and should not handle any logic (deciding what to display). These two should be separated into `renderSprite(SpriteBatch batch)` and `renderAlertIndicator(SpriteBatch batch)`, with `render()` calling both. It also makes each piece independently testable and readable.

Smell 4: Data Clump / Magic Numbers

Files: activateMap() in GameScreen.java **Lines:** 1016-1060 **Commit:** 3c4a96a

In activateMap(), police and puddle enemies are spawned using long lists of hard-coded coordinates with no explanation of what each position represents.

```
police.add(new PoliceEnemy(500f, mapYOffsets[2] + 850f));
police.add(new PoliceEnemy(1300f, mapYOffsets[2] + 1200f));
// ... 18 more
```

There are 20 police spawns and 13 puddle spawns for map 3 alone, all as float numbers. This is a data clump, where a group of values that always appear together but are not encapsulated. If the map layout changes, every coordinate must be searched for and updated manually.

The fix is to extract the coordinates into named constant arrays at the top of GameScreen.java (e.g. MAP3_POLICE_SPAWNS, MAP3_PUDDLE_SPAWNS), and create a helper method spawnEnemiesForMap(int index) that iterates over the arrays. activateMap() then calls spawnEnemiesForMap(index), making it shorter and easier to maintain.

```
private static final float[][] MAP3_POLICE_SPAWNS = {
    {500f, 850f}, {1300f, 1200f}, {2500f, 1300f}, ...
};
```

Refactoring (Based on Megan's Review)

After receiving Megan's code review, I refactored each identified code smell as follows:

Smell 1: Unnecessary Variable **Commit:** fd23f22

bikeParked was removed from GameScreen.java. All assignments to bikeParked were deleted, and the one usage if (bikeParked) in the render loop was replaced with if (!isOnBike). The game was rerun and bike parking, mounting, and dismounting all ran normally. All existing tests passed.

```
// Before
bikeParked = true;
isOnBike = false;
// After
isOnBike = false; // bikeParked removed; use !isOnBike instead
```

Smell 2: Data Clump **Commit:** e0d5641

In EndScreen.java, the DIGIT_ and TIME_ constants were grouped into a DigitLayout structure so that related values (position, size, gap) are encapsulated together rather than stored as separate unrelated fields. This also reduced the amount of similar code definitions in the beginning. The end screen was tested and displayed correctly after the change. All existing tests passed.

```
// Before
private static final float DIGIT_X = 348f, DIGIT_Y = 460f;
private static final float DIGIT_W = 150f, DIGIT_H = 140f, DIGIT_GAP = 139f; // ... repeat for all
adjustments and TIME_
// After
private static final DigitLayout SCORE_LAYOUT = new DigitLayout(348f, 460f, 150f, 140f, 139f);
private static final DigitLayout TIME_LAYOUT = new DigitLayout(450f, 328f, 50f, 50f, 35f);
```

Smell 3: Code Duplication **Commit:** b39dd53

In EndScreen.java, the repeated digit-extraction pattern (/10, %10) shared by scoreDigits and timeDigits was extracted into a private helper method digitsOf(). Both arrays now call the shared method instead of repeating the same operations over and over. The end screen was tested and displayed correct score and time values.

```
// Before: repeated in both scoreDigits and timeDigits
(s / 1000) % 10, (s / 100) % 10, ...
// After: shared helper
private int[] digitsOf(int value, int count) {
    int[] d = new int[count];
    for (int i = count - 1; i >= 0; i--) {
        d[i] = value % 10;
        value /= 10;
    } return d;
}
```

Smell 4: Dead Code **Commit:** 26ee304

In House.java, the unused import, import com.badlogic.gdx.Gdx, was removed. No behaviour changed as the import was never referenced. The project was rebuilt successfully, and all existing tests passed.